

miniFlink: руководство для начала работы

Сквозной разбор от потока-функции до durable-пайплайна

Проект miniFlink

Аннотация

Это пошаговое руководство для тех, кто впервые видит miniFlink. Мы строим один пайплайн от начала до конца на сквозном примере — телеметрии температурных датчиков — и на каждом шаге добавляем ровно одно понятие: поток, событие и время, watermark, ключ группировки, окна, агрегацию, stateful-правила, дедупликацию и, наконец, персистентность. Все листинги взяты из компилируемого кода. Прочитав руководство сверху вниз, вы сможете читать и писать пайплайны miniFlink и будете знать, куда смотреть дальше. Примеры по возрастанию сложности живут в каталоге `examples/` (`ex01-ex10`); это руководство — сквозная нить, связывающая их.

Содержание

1 Введение: зачем нужна потоковая обработка	1
1.1 Два способа работать с данными	1
1.2 Что такое «поток»	2
1.3 Почему не просто запрос к базе раз в час?	2
1.4 Почему не просто Kafka или «напишу сам»?	2
1.5 Что даёт потоковый движок	3
1.6 Когда пакетная обработка — правильный выбор	3
1.7 Как читать дальше	4
2 Прежде чем начать	4
3 Шаг 1. Поток как функция	4
4 Шаг 2. Событие и время	5
5 Шаг 3. Watermarks	5
6 Шаг 4. Ключ группировки через KEYED	5
7 Шаг 5. Окна и агрегация	6
8 Шаг 6. Stateful-правила: триггеры	6
9 Шаг 7. Дедупликация поздних дублей	7
10 Ещё операторы для реальных систем	7
10.1 Обогащение из справочника: <code>enrich</code>	7
10.2 Окна по паузам: <code>session_window</code>	8
10.3 Объединение потоков: <code>keyed_join</code>	8
10.4 Возраст молчания: <code>silence_age</code>	8

11 Порог из данных, а не из кода	9
11.1 Способ 1: обогащение из таблицы порогов	9
11.2 Способ 2: слияние потоков и состояние по зоне	10
11.3 Способ 3: «порог на момент события» (temporal join)	11
11.4 Две тонкости, о которых легко забыть	12
12 Шаг 8. Поздние данные: retract и update	13
13 Шаг 9. Персистентность без изменения пайплайна	13
14 Шаг 10. Масштабирование и exactly-once	14
15 Что дальше	15

1. Введение: зачем нужна потоковая обработка

Этот раздел не про код. Он объясняет простыми словами, что такое потоковая обработка, какую задачу она решает и почему для некоторых задач она удобнее привычных подходов — запросов к базе данных или самостоятельно написанной программы-обработчика. Если вы никогда не слышали этих слов — ничего страшного, начнём с нуля.

1.1. Два способа работать с данными

Представьте, что вы следите за своим банковским счётом. Узнать, что с деньгами что-то не так, можно двумя способами.

Способ первый. В конце месяца приходит выписка. Вы садитесь и просматриваете все операции разом. Картина полная и аккуратная: все траты, итоги, сравнение с прошлым месяцем. Но если в середине месяца кто-то украл карту и потратил деньги, вы узнаете об этом только сейчас — спустя недели, когда деньги уже ушли.

Способ второй. В ту самую секунду, когда происходит подозрительная операция, вам приходит сообщение: «с вашей карты пытаются снять крупную сумму в другом городе». Вы реагируете немедленно, пока ничего ещё не случилось.

Это не «хороший» и «плохой» способы. Это два разных инструмента для двух разных задач. Первый — *пакетная* обработка (по-английски *batch*): данные сначала накапливаются, а потом обрабатываются все сразу. Второй — *потоковая* обработка: каждое событие обрабатывается в тот момент, когда оно происходит, не дожидаясь остальных.

1.2. Что такое «поток»

Слово *поток* здесь стоит понимать буквально — как поток воды в реке. У реки нет «конца», который можно дожидаться: вода течёт постоянно. Точно так же во многих системах данные приходят непрерывно: датчик каждую секунду присылает температуру, касса пробивает чек за чеком, пользователи кликают по сайту круглые сутки. Каждую такую «каплю» — одно показание, один чек, один клик — мы будем называть *событием*.

Пакетный подход говорит: «накопим события за час (или за день), потом обработаем». Поточковый подход говорит: «обработаем каждое событие, как

только оно появилось, и сразу обновим результат». Разница — как между «посмотреть утром вчерашнюю запись с камеры» и «охранник смотрит на мониторы прямо сейчас и реагирует на ходу».

1.3. Почему не просто запрос к базе раз в час?

Самый привычный способ работать с данными — сложить их в базу и писать запросы (обычно на языке SQL): «покажи среднюю температуру за последний час», «посчитай продажи за день». Для отчётов это прекрасно. Но если задача — *реагировать вовремя*, у такого подхода есть три слабых места.

Задержка. Если запрос запускается раз в час, то о проблеме вы узнаете в среднем через полчаса после того, как она случилась. Для отчёта это неважно. Для тревоги о перегреве оборудования или утечке газа — полчаса может быть слишком поздно.

Лишняя работа. Каждый раз запрос пересчитывает всё заново — перебирает все накопившиеся данные, даже если изменилась всего пара строк. Чем больше данных, тем тяжелее и дороже каждый прогон. Поток же обрабатывает только новое событие и просто *подправляет* уже посчитанный результат.

Самое тонкое — время. У каждого события есть два разных момента: когда оно *произошло на самом деле* и когда оно *дошло до вас*. В реальности данные опаздывают и приходят не по порядку: датчик в шахте мог измерить температуру в 14:00, но из-за плохой связи его показание добралось до сервера только в 14:05, уже после более свежих данных. Если считать «по времени получения», легко сделать неверный вывод — например, «датчик молчал пять минут», хотя он просто не мог достучаться. Поточковые системы умеют рассуждать о времени *события*, а не о времени его получения, и аккуратно разбираться с опоздавшими данными. Сделать это правильно вручную — на удивление сложно.

1.4. Почему не просто Kafka или «напишу сам»?

Здесь часто звучит слово *Kafka*. Полезно понимать, что это такое и чем оно *не* является.

Kafka — это, если по-простому, очень надёжный конвейер (или почта) для сообщений. Одна программа кладёт события на конвейер, другая — снимает их с другого конца. Kafka гарантирует, что сообщения не потеряются и дойдут по порядку. Это отличный и важный инструмент — но он занимается *перевозкой* событий, а не их *осмыслением*. Kafka сам по себе не знает, что такое «среднее за минуту», не умеет группировать события в окна времени, не отличает дубликат от нового события и не подскажет, что делать с опоздавшим показанием.

Значит, всю эту логику кто-то должен написать в программе, которая читает события с конвейера. И вот тут кроется главная трудность. Вещи, которые на словах звучат просто, на практике полны подводных камней:

- считать «среднее за последнюю минуту» — нужно решить, что такое «минута», когда её закрывать и что делать с опоздавшими;
- не посчитать одно и то же дважды, если событие случайно пришло по конвейеру два раза;
- не сломаться, когда события приходят не по порядку;

- не потерять накопленные итоги, если программа перезапустилась (упала и поднялась снова).

Каждый из этих пунктов — источник тонких, незаметных ошибок, которые проявляются редко и в самый неудачный момент. Написанные руками с нуля, они почти наверняка где-то будут неверны.

1.5. Что даёт потоковый движок

Потоковый движок (Apache Flink — самый известный; miniFlink — компактная реализация той же идеи) — это набор готовых, проверенных «кирпичиков» ровно для этих задач: окна времени, подсчёты и средние, правила «подними тревогу, когда значение перешло порог», защита от дубликатов, аккуратная обработка опоздавших данных и сохранение состояния при перезапуске. Вы не пишете эту механику заново — вы *собираете* пайплайн из кирпичиков, а сложные и опасные детали уже решены внутри и покрыты тестами.

Важно и то, как это выглядит. Хорошо собранный потоковый пайплайн читается почти как описание задачи на человеческом языке: «сгруппируй по датчику, посчитай среднее за минуту, подними тревогу при перегреве». Не «как именно перебрать данные», а «что мы хотим получить». Именно к этому — чтобы код читался как спецификация — и стремится miniFlink. Дальше в руководстве вы увидите, что весь пайплайн — это короткая цепочка таких шагов.

1.6. Когда пакетная обработка — правильный выбор

Будет честно сказать и обратное. Потоковая обработка — не замена всему. Если задача — ночной отчёт, пересчёт зарплат раз в месяц, аналитика по историческим данным или разовая выгрузка — пакетный подход и обычный SQL проще, понятнее и полностью достаточны. Реагировать «прямо сейчас» там не нужно, а полная картина важнее скорости.

Потоковая обработка выигрывает там, где совпадают три вещи: данные поступают *непрерывно*, реагировать нужно *быстро*, и важно *время самих событий* (а также корректность при опозданиях, дубликатах и перезапусках). Мониторинг оборудования, обнаружение мошенничества, живые дашборды, оповещения о безопасности — классические примеры. Система мониторинга шахты, на операторах которой построено это руководство, — ровно такой случай.

1.7. Как читать дальше

Дальше мы построим один такой пайплайн от начала до конца на сквозном примере — показаниях температурных датчиков. Каждый следующий раздел добавляет ровно одно понятие из перечисленных выше: сначала «поток», потом «событие и время», потом окна, тревоги, защиту от дубликатов, обработку опоздавших данных и, наконец, сохранение состояния. К концу вы сможете читать и писать такие пайплайны сами. Технических терминов бояться не нужно — каждый вводится по одному и сразу на примере.

2. Прежде чем начать

miniFlink — это библиотека потоковой обработки на OCaml, реализующая ядро семантики Apache Flink на одном узле: событийное время, watermarks, окна, stateful-операторы, retract/update для поздних данных и exactly-once. Никакого брокера, планировщика или фонового цикла; весь пайплайн — это композиция обычных функций, которую можно читать как спецификацию.

Чтобы запускать примеры, нужен OCaml (4.14+ или 5.x) и Dune 3.0+. Соберите проект командой `dune build` и прогоните тесты — `dune test`. Каждый листинг ниже предполагает, что в начале файла стоит `open Miniflink`.

Сквозной пример. На протяжении всего руководства мы обрабатываем поток показаний температурных датчиков и хотим в итоге: считать среднюю температуру по датчику в окне, поднимать тревогу при перегреве и переживать перезапуск процесса без потери состояния. Доменная модель проста:

```
type reading = { sensor : string; celsius : float }
```

3. Шаг 1. Поток как функция

Центральная абстракция — `Stream.t`, и она устроена предельно просто:

```
type 'a t = unit -> 'a option
```

Поток — это функция без аргументов, которая при каждом вызове возвращает либо следующий элемент (`Some x`), либо `None`, если элементы кончились. Это *pull*-модель: ничего не происходит, пока потребитель не «потянет» следующее значение. Весь граф обработки — это стопка таких функций, обёрнутых одна вокруг другой.

```
let nats : int Stream.t =
  let i = ref 0 in
  fun () -> let v = !i in incr i; if v < 5 then Some v else None

let doubled = Stream.map (fun x -> x * 2) nats
```

`Stream.map`, `Stream.filter` и `Stream.flat_map` — это операторы преобразования: каждый принимает поток и возвращает новый поток, не вычисляя ничего заранее. Композиция через оператор `|>` читается естественно слева направо.

4. Шаг 2. Событие и время

Голый поток значений ничего не знает о времени. В потоковой обработке время — первоклассное понятие, поэтому miniFlink оборачивает значения в `Mf_event.t` с четырьмя вариантами:

- `Data (value, ts)` — значение с временной меткой (event-time);
- `Watermark ts` — отметка прогресса времени (см. шаг 3);
- `Retract (value, ts)` — отмена ранее выданного результата;

- `Update {old; new_value; ts}` — атомарная замена старого результата **НОВЫМ**.

Последние два понадобятся, когда мы дойдём до поздних данных; пока что работаем с `Data`. Превратим наши показания в `event-time` поток — пусть каждое следующее чтение приходит на секунду позже:

```
let event_stream : reading Mf_event.t Stream.t =
  readings
  |> List.mapi (fun i r -> Mf_event.data r (i * 1000))
  |> Stream.of_list
```

Временные метки здесь в миллисекундах; `Time` — это просто `int` мс с удобными конструкторами (`Time.seconds`, `Time.minutes`).

5. Шаг 3. Watermarks

Событийное время порождает вопрос: когда можно считать, что «все события до момента t уже пришли»? В распределённой системе данные опаздывают и приходят не по порядку. *Watermark* — это обещание системы: «событий с меткой меньше t больше не будет». Операторы окон используют его, чтобы понять, когда окно можно закрывать и эмитить результат.

```
let with_wm =
  Mf_event.with_watermarks ~latency:(Time.seconds 1) event_stream
```

Параметр `~latency` задаёт допуск на опоздание: `watermark` отстаёт от максимальной увиденной метки на эту величину, что оставляет опоздавшим событиям шанс прийти. Чем больше `latency`, тем устойчивее к опозданиям, но тем позже закрываются окна — это прямой компромисс между полнотой и задержкой.

6. Шаг 4. Ключ группировки через KEYED

Почти все интересные операторы работают *по ключу*: окно «на датчик», агрегат «на пользователя». В большинстве фреймворков ключ передаётся параметром в каждый оператор. `miniFlink` кодирует ключ *один раз* в типе через `first-class`-модуль `Keyed.S`:

```
module By_sensor : Keyed.S with type t = reading = struct
  type t = reading
  let key r = r.sensor
end
```

Теперь `By_sensor` можно передавать операторам как значение, и им не нужен отдельный параметр `~key`. Для разовых случаев есть `Keyed.of_fun` (`fun r -> r.sensor`), не требующий объявления модуля. Этот приём и делает пайплайны похожими на спецификацию: ключ виден один раз, а не повторяется в каждом вызове.

7. Шаг 5. Окна и агрегация

Окно режет бесконечный поток на конечные куски, к которым можно применить агрегат. Простейшее — *тумблинг*: неперекрывающиеся окна фиксированного размера. Посчитаем среднюю температуру по каждому датчику в окнах по три секунды:

```
let avg_per_sensor =
  event_stream
  |> Pipe.window_agg (module By_sensor)
    (Pipe.tumbling (Time.seconds 3))
    (Agg.mean (fun r -> r.celsius))
```

`Pipe.window_agg` принимает модуль ключа, спецификацию окна (`tumbling`, `sliding`, и т. д.) и *агрегат*. Агрегаты — это композируемая библиотека: `count`, `sum`, `mean`, `count_if`, а также комбинаторы вроде `both` (считать два агрегата сразу) и `contraMap` (адаптировать входной тип). На выходе — поток (`string * 'r`): ключ и результат агрегата за закрытое окно. Результат появляется, когда `watermark` пересекает границу окна — вот зачем был нужен шаг 3.

8. Шаг 6. Stateful-правила: триггеры

Окна отвечают на «сколько/какое среднее», но для системы оповещения нужен *stateful*-автомат: поднять тревогу, когда значение пересекает порог, и снять её, когда вернулось в норму, не дёргая оператора на каждом событии. Это `Trigger`:

```
let alert_spec =
  Trigger.create
    ~name: "overheat"
    ~condition: (Trigger.greater_than 90.0)
    ~produce_alert: (fun ~key ~value ~ts: _ ->
      Printf.sprintf "ALERT %s=%.0f" key value)
    ~produce_recovery: (fun ~key ~ts: _ ->
      Printf.sprintf "OK %s" key)
    ()

let alerts =
  event_stream
  |> Stream.map (Mf_event.map_value (fun r -> (r.sensor, r.celsius)))
  |> Trigger.of_stream alert_spec
```

Триггер — это конечный автомат на ключ. Когда температура датчика превышает 90, он эмитит `Data` с алертом; когда падает обратно — `Retract` прежнего алерта и `Data` с `recovery`. Можно задать `debounce` (`~problem_for`), гистерезис (`greater_than_with`) и произвольные условия (`custom`). Важное свойство, проверенное `property`-тестами: активных алертов на ключ не больше одного, а каждый `recovery` парен своему алерту — автомат не «залипает».

9. Шаг 7. Дедупликация поздних дублей

Семантика `at-least-once` у источников означает дубли: один пакет может прийти дважды. `Pipe.dedup` подавляет повторы с одним ключом в пределах

окна cooldown:

```
module By_pair : Keyed.S with type t = string * float = struct
  type t = string * float
  let key (k, _) = k
end

let deduped =
  event_stream
  |> Stream.map (Mf_event.map_value (fun r -> (r.sensor, r.celsius)))
  |> Pipe.dedup (module By_pair)
    ~rule:(fun (k, _) -> k)
    ~cooldown:(Time.seconds 5)
```

Свойство, зафиксированное property-тестом: между двумя прошедшими событиями одного ключа разрыв по времени не меньше `cooldown`, и каждое прошедшее событие действительно было во входе.

10. Ещё операторы для реальных систем

Шаги 1-7 — это костяк. Но в настоящей системе мониторинга (как `minePASS`, на котором основан сквозной пример) почти всегда нужны ещё четыре приёма: дополнить событие справочными данными, сгруппировать всплеск активности, объединить несколько потоков и заметить, что источник *замолчал*. Все они — такие же операторы, как уже знакомые.

10.1. Обогащение из справочника: `enrich`

Событие часто несёт только идентификатор — например, номер датчика — а человеку нужно человекочитаемое: где этот датчик стоит. Такие справочные данные («датчик А — штрек 3») живут в *таблице*, и `Pipe.enrich` подмешивает их в проходящие события по ключу:

```
let dict = Table.of_list [ ("A", "Штрек 3"); ("B", "Ствол") ]

let enriched =
  event_stream
  |> Pipe.enrich (module By_sensor)
    ~from:dict
    ~merge:(fun r loc -> match loc with
      | Some location -> { r with location }
      | None -> r)
```

`~from` — таблица-справочник, `~merge` — как вписать найденное значение (или его отсутствие) в событие. Таблицу можно собрать из списка (`Table.of_list`), из `Hashtbl` или из *другого потока* (`Table.build`) — тогда справочник обновляется на лету по мере прихода новых данных.

10.2. Окна по паузам: `session_window`

Тумблинг-окна из шага 5 режут время на равные куски. Но иногда естественная единица — не «минута», а *всплеск активности*: группа событий, идущих подряд, отделённая от следующей группы паузой. Например, серия показаний, пока

техника работает, затем затишье, затем новая серия. `session_window` закрывает окно, когда после события прошло больше `~gap` тишины:

```
let sessions =  
  event_stream  
  |> Pipe.session_window (module By_sensor) ~gap:(Time.seconds 5)
```

На выходе — (ключ, список событий сессии). Если два события одного датчика разделены паузой больше `gap`, они попадут в разные сессии; если меньше — в одну. Размер окна, таким образом, определяют сами данные, а не часы.

10.3. Объединение потоков: `keyed_join`

Часто про один объект приходят разные данные из разных источников: температура из одного потока, влажность — из другого, вибрация — из третьего. `keyed_join` сводит несколько потоков в один, сопоставляя события по ключу:

```
let joined =  
  Pipe.keyed_join (module By_sensor) [ temp_stream; humidity_stream ]
```

На каждое новое событие из любого входного потока на выходе появляется (ключ, список последних значений из каждого потока) — где ещё не пришедшие значения отмечены как «пусто». Так в одном месте собирается полная свежая картина по объекту, даже если источники обновляются вразнобой.

10.4. Возраст молчания: `silence_age`

Для безопасности часто опаснее не «плохое значение», а *тишина*: датчик, который перестал присылать данные, может означать обрыв связи или поломку. `Item.silence_age` следит за каждым ключом и сообщает, сколько времени прошло с его последнего события:

```
let silence =  
  event_stream  
  |> Item.silence_age ~by:(fun r -> r.sensor) ~tick:(Time.seconds 10)
```

На каждое реальное событие он эмитит (ключ, 0) — «возраст обнулился», — а затем каждые `~tick` миллисекунд тишины — (ключ, возраст) с растущим числом. Этот поток дальше можно подать в триггер из шага 6: «подними тревогу, если датчик молчит дольше минуты». Получается оповещение об *отсутствии* данных — то, что `batch`-запрос увидел бы в лучшем случае с большим опозданием.

Прямее: `on_silence`. `silence_age` даёт *поток возраста*, который надо соединять с триггером. Если нужен сразу алерт об отсутствии, есть более прямой оператор — `Pipe.on_silence`: он сам эмитит событие, когда признак не наблюдался дольше порога, и (опционально) `recover` при возобновлении.

```
packets  
|> Pipe.event_time ~lateness:(seconds 1)  
|> Pipe.on_silence (module ByLamp)  
  ~within:(minutes 2)  
  ~has:(fun _ -> true) (* любой пакет = жив<<>> *)  
  ~on_silent:(fun key ~last ~ts -> No_signal (key, last))
```

```
~on_resumed:(fun key ~ts -> Signal_back key)
```

Время перехода в тишину — `last + within` (когда молчание превысило порог), а не время `watermark`, который это обнаружил. `~has` выбирает, что считается «признаком жизни»: любой пакет, только пакет с показаниями, только с движением. Когда один детектор должен молчать, пока активен другой, их композитрует `Pipe.suppress_while`. Оба выросли из реального кода `ex07` (см. ниже раздел про операторы за пределами линейного pipeline).

11. Порог из данных, а не из кода

В шаге 6 порог тревоги был *вшит* прямо в код:

```
~condition:(Trigger.greater_than 90.0)
```

Число 90 написано раз и навсегда. Но в реальной системе пороги меняются: для разных зон они разные, оператор хочет подкручивать их на ходу, не перезапуская программу. Хочется, чтобы порог был *настройкой*, а не строчкой в коде.

Идея простая: пусть порог станет таким же *данным*, как и само показание. Тогда условие будет сравнивать два числа, которые пришли вместе с событием. Заведём для этого тип, который несёт и замер, и действующий порог:

```
type checked = { zone : string; celsius : float; max_celsius : float }
```

И условие триггера теперь смотрит *внутри* значения, а не на зашитую константу — для этого есть `Trigger.custom`, который принимает два произвольных предиката:

```
let condition =  
  Trigger.custom  
    ~problem:(fun c -> c.celsius > c.max_celsius)  
    ~recovery:(fun c -> c.celsius <= c.max_celsius)
```

Всё остальное в триггере не меняется: он по-прежнему ведёт отдельный автомат на зону, держит не больше одной активной тревоги и парит `recovery` к тревоге. Изменилось только одно — порог переехал из кода в данные. Остаётся вопрос: *откуда* в событии берётся поле `max_celsius`? Рассмотрим два способа, от простого к правильному.

11.1. Способ 1: обогащение из таблицы порогов

Первый способ опирается на уже знакомый `enrich` (см. предыдущий раздел). Пороги хранятся в таблице «зона → порог», а `enrich` подмешивает текущий порог в каждое проходящее показание:

```
let thresholds = Table.of_list [ ("A", 90.); ("B", 80.) ]  
  
let alerts =  
  readings  
  (* пока порог неизвестен, кладем заведомо безопасное значение *)  
  |> Stream.map (Mf_event.map_value (fun r ->  
    { zone = r.zone; celsius = r.celsius; max_celsius = infinity }))  
  |> Pipe.enrich (module Checked_by_zone)  
    ~from:thresholds
```

```

~merge:(fun c m -> match m with
  | Some max_celsius -> { c with max_celsius }
  | None -> c)
|> Stream.map (Mf_event.map_value (fun c -> (c.zone, c)))
|> Trigger.of_stream spec

```

Таблицу можно собрать не из списка, а *из потока* порогов (`Table.build`) — тогда она будет обновляться сама, как только оператор пришлёт новое значение. Это и есть «динамическая таблица уставок».

На что обратить внимание. Два момента, важных именно для тревог.

Во-первых, случай `None` в `~merge` — это ситуация «для зоны порог ещё не задан». Что тут правильно, решаете вы: в примере выше мы подставляем `infinity` (без порога тревоги не будет — безопасный по умолчанию выбор для «больше чем»), но в иной системе разумнее, наоборот, поднять тревогу «зона без настройки». Молча проигнорировать этот случай — плохая идея.

Во-вторых, у этого способа есть тонкость со временем. `enrich` берёт *то значение, что лежит в таблице прямо сейчас*. Если порог и показание приходят вразнобой, показание может проверяться против ещё не обновлённого порога. Для отчётов это незаметно, для тревог — иногда важно. Поэтому есть второй способ.

11.2. Способ 2: слияние потоков и состояние по зоне

Более аккуратный подход — относиться к порогам как к ещё одному *потоку событий*, наравне с показаниями, и свести оба потока в один. Тогда у нас одна лента, упорядоченная по времени, где перемешаны два вида событий: «пришло новое показание» и «оператор сменил порог».

Сначала опишем, что входом может быть одно из двух:

```
type input = Reading of reading | Setpoint of setpoint
```

Сольём оба потока в один по времени с помощью `Mf_event.union`:

```

let merged =
  Mf_event.union
    (Stream.map (Mf_event.map_value (fun r -> Reading r)) readings)
    (Stream.map (Mf_event.map_value (fun s -> Setpoint s)) setpoints)

```

Теперь обработаем эту общую ленту *stateful*-оператором `process_keyed` (см. под капотом он же стоит за триггером). На каждую зону он держит маленькое состояние — текущий порог — и обновляет его, когда приходит новый порог, либо сравнивает с ним, когда приходит показание:

```

let alerts =
  merged
  |> Pipe.process_keyed (module Input_by_zone)
    ~init:(fun () -> { max_c = None }) (* порог ещё не задан *)
    ~on_event:(fun ctx zone st input ->
      match input with
      | Setpoint s ->
        st.max_c <- Some s.max_celsius (* запомнили новый порог *)
      | Reading r ->
        match st.max_c with
        | Some m when r.celsius > m ->

```

```

        ctx.emit (alert zone r.celsius m)
    | _ -> ()
    ~on_timer:(fun _ _ _ _ -> ())

```

У этого способа есть важное преимущество, которого нет у первого. Состояние пересматривается на *любое* входное событие — в том числе на смену порога. Представьте: температура 85, порог 90 — тихо. Оператор снижает порог до 80. Те же 85 теперь должны стать тревогой *немедленно*, не дожидаясь нового замера. Первый способ (обогащение) этого не заметит, пока не придёт следующее показание — триггер видит только поток показаний. А здесь смена порога — это полноценное событие на ленте, и оператор может тут же переоценить ситуацию. Для тревог это часто решающий довод.

11.3. Способ 3: «порог на момент события» (temporal join)

У способов 1 и 2 есть общая слепая зона. Оба отвечают на вопрос «какой порог *сейчас*». Но представьте, что само показание *опоздало*: датчик измерил температуру в 14:00, а из-за плохой связи показание дошло до нас только в 14:05. И пусть за эти пять минут оператор успел сменить порог. Против какого порога правильно проверять это показание — того, что действовал в 14:00, когда замер был сделан, или нынешнего? Для честного разбора инцидента нужен первый — порог, *актуальный на момент самого события*.

Именно это делает Temporal.temporal_join. Он хранит не одно текущее значение порога на зону, а всю их *историю* («с 13:00 порог был 90, с 14:02 стал 80»), и для каждого показания подставляет ту версию, что действовала в его event-time:

```

let alerts =
  readings
  |> Stream.map (Mf_event.map_value (fun r ->
    { zone = r.zone; celsius = r.celsius; max_celsius = infinity }))
  |> Temporal.temporal_join
    ~key_main:(fun c -> c.zone)          (* ключ стороны показаний *)
    ~key_upd:(fun s -> s.sp_zone)        (* ключ стороны порогов *)
    ~valid_from:(fun s -> s.since)      (* с какого момента порог в силе *)
    ~merge:(fun c sp -> match sp with
      | Some s -> { c with max_celsius = s.value }
      | None -> c)
    ~updates:setpoints
  |> Stream.map (Mf_event.map_value (fun c -> (c.zone, c)))
  |> Trigger.of_stream spec

```

Ключевая деталь — в том, как он обращается со временем. Оба потока (показания и пороги) несут watermarks (шаг 3). Когда приходит показание с временем T , оператор *придерживает* его, пока watermark потока порогов не дойдёт до T — то есть пока не станет гарантировано, что все пороги, которые могли вступить в силу к моменту T , уже получены (даже те, что пришли с опозданием). Только тогда показание обогащается нужной версией и идёт дальше. Благодаря этому результат *не зависит* от порядка прихода: опоздавший порог встаёт на своё место в истории, а показание дожидается его и проверяется правильно.

Чем платим. За эту корректность есть две цены, и о них честно стоит знать. Во-первых, *задержка*: показание не выходит сразу, а ждёт, пока «подтянется» время порогов — это плата за правильность под опозданиями. Во-вторых, *память*: хранится вся история версий порога, и без обрезки она растёт без конца. Для этого есть `Temporal.prune_versions_before` — он отрезает версии, которые уже никакое будущее показание не запросит. Вызывать его стоит, когда показания гарантированно ушли вперёд по времени.

Какой способ выбрать. Короткий ориентир. Способ 1 (обогащение) — когда пороги меняются редко, а строгая точность по времени не критична: проще всего. Способ 2 (слияние и состояние) — когда важно *мгновенно* переоценивать ситуацию при смене порога и хватает «текущего» значения. Способ 3 (`temporal join`) — когда данные *опаздывают*, а порог за это время мог меняться, и нужно проверять каждое показание против порога, действовавшего *именно в его момент*. Для большинства задач достаточно первых двух; третий — для случаев, где важна точность разбора задним числом.

11.4. Две тонкости, о которых легко забыть

Поток порогов почти всегда молчит. Показания идут потоком, а порог оператор меняет раз в день. «Молчащий» поток порогов опасен для событийного времени: пока по нему ничего не приходит, его `watermark` (шаг 3) стоит на месте и тормозит весь объединённый пайплайн — тревоги по показаниям замирают в ожидании. Лекарство — продвигать время такого редкого потока по настенным часам через `Mf_event.with_idle_watermarks`: «если из источника давно ничего не было, считаем, что время всё равно идёт».

Гистерезис тоже можно вынести в данные. В шаге 6 был встроенный гистерезис (поднять тревогу при одном значении, снять при другом, чтобы не дребезжало у самого порога), но с фиксированными числами. Через `custom` его легко сделать настраиваемым: пусть порог-событие несёт *два* числа — порог тревоги и порог снятия:

```
~problem: (fun c -> c.celsius > c.alarm)
~recovery: (fun c -> c.celsius < c.clear)
```

Тогда и ширину гистерезиса можно настраивать на каждую зону и менять на ходу — то, чего фиксированная версия не умела.

Главное, что стоит унести из этого раздела: как только порог становится данными, а не константой, открывается всё то же, что мы умеем делать с данными — хранить, обновлять потоком, объединять, переоценивать. Логика тревоги при этом остаётся такой же короткой и читаемой.

12. Шаг 8. Поздние данные: `retract` и `update`

Вернёмся к четырёхвариантному событию из шага 2. Что, если событие опоздало и пришло уже после закрытия его окна? Наивный вариант — выбросить его (потеря данных) или выдать новый результат поверх старого (дашборд «мигает» промежуточным состоянием). `miniFlink` делает третье: атомарно *корректирует* результат.

Когда в уже закрытое окно приходит позднее значение, оператор эмитит `Update(o → n)` — одно событие, заменяющее старый результат *o* новым *n* без промежуточного состояния. Для потребителя это выглядит как мгновенный переход, без «мигания». Фундаментальный закон, который мы проверяем property-тестами для всех stateless-операторов:

$$\text{Update}(o \rightarrow n) \equiv \text{Retract}(o) \text{ затем } \text{Data}(n),$$

то есть атомарный `Update` — это в точности пара «отозвать старое, выдать новое», только без промежуточного шага. Писать обработку `Retract/Update` вручную обычно не нужно: операторы (`map`, `filter`, `window_agg`, `trigger`) делают это сами и согласованно.

Когда нужна своя логика на ключ. Иногда доменная логика не укладывается в готовые операторы: нужно держать состояние на ключ и самому решать, когда выдать `Data`, когда *отозвать* (`Retract`) уже выпущенный результат, а когда *обновить* (`Update`). Для этого есть `Pipe.process_keyed`: состояние на ключ держит библиотека, а контекст `ctx` даёт `emit`, `emit_retract` и `emit_update` — те же три конструктора, но под вашим контролем.

```
events
|> Pipe.process_keyed (module ByKey)
  ~init:(fun () -> { current = None })
  ~on_event:(fun ctx key st v ->
    match st.current with
    | None      -> st.current <- Some v; ctx.emit v
    | Some old -> st.current <- Some v; ctx.emit_update ~old v)
  ~on_timer:(fun _ _ _ _ _ -> ())
```

А когда на одном ключе сходятся несколько потоков разных типов (например `driver` + справочник, приходящий отдельным потоком), `Pipe.co_process2/co_process3` держат общее состояние на ключ и вызывают свой обработчик на каждый вход. Оба выросли из реального кода `ex07`; полный разбор — в `docs/expressiveness.md` (раздел 4) и в примере `examples/ex07_location/`.

Ещё одна практичная деталь для продакшена: `process_keyed` и `window_fold` принимают `~?on_error` — если ваш колбэк бросит исключение на «ядовитом» событии, оно уйдёт в обработчик (например в `DLQ`), а пайплайн продолжит работать, а не упадёт.

13. Шаг 9. Персистентность без изменения пайплайна

Финальное требование: пережить перезапуск процесса. Активная тревога не должна исчезнуть из-за рестарта — иначе диспетчер увидит мнимое снятие.

Ключевая идея `miniFlink` — *ортогональная* персистентность: один и тот же пайплайн работает с `durability` и без неё, режим выбирается *снаружи*, а не вшит в операторы. Сначала пишем пайплайн как обычно:

```
let pipeline src =
  src
  |> Stream.map (Mf_event.map_value (fun r -> (r.sensor, r.celsius)))
  |> Trigger.of_stream alert_spec
```

Без персистентности — просто запускаем. С персистентностью — оборачиваем тот же вызов в durable-контекст:

```
(* впамяти , ничегонесохраняется *)
let ephemeral () = pipeline event_stream

(* тожесамое , носостояние durable *)
let durable () =
  let backend = Persistence_backend.of_memory (Hashtbl.create 16) in
  Runtime_context.with_context
    (Runtime_context.durable backend)
    (fun () -> pipeline event_stream)
```

Пайплайн не изменился ни на строку — разница только в обёртке `with_context`. Сериализация тоже ортогональна: состояние операторов сериализуется по умолчанию (через `Marshal`), так что для `crash-recovery` не нужно писать ни одного кодека; для эволюции схемы между деплоями можно задать реестр кодеков в контексте, опять же не трогая пайплайн. В продакшене вместо `in-memory backend` подставляется `RocksDB`; пайплайн при этом снова не меняется.

Это свойство — что `durable`-прогон с «крашем» посередине даёт тот же результат, что непрерывный — мы проверяем `property`-тестом на сотнях случайных потоков и точек краша.

14. Шаг 10. Масштабирование и *exactly-once*

Всё выше — однопоточный пайплайн. Для пропускной способности `Parallel.run_parallel_simple` шардирует поток по ключу на N воркеров. Когда нужна не просто параллельность, а *exactly-once* с восстановлением после сбоя, используется `Checkpoint_parallel.run_exactly_once`: координатор инжектирует `barrier`-маркеры (снимок в стиле Chandy-Lamport), каждый воркер снапшотит своё состояние, а `checkpoint` фиксируется атомарно, когда снимки собраны со всех воркеров. Транзакционный `sink` и восстановление `source` по `offset` замыкают *exactly-once* на одном узле.

```
Checkpoint_parallel.run_exactly_once
  ~workers:4 ~capacity:256 ~checkpoint_every:1000
  ~key_of ~make_state ~process ~source ~sink ~store ()
```

Это другой, более высокий уровень гарантий, чем `per-operator` персистентность из шага 9: тот отвечает за восстановление состояния оператора, этот — за `end-to-end exactly-once` со стороны источника и приёмника. Выбирайте по задаче: для большинства пайплайнов достаточно ортогональной персистентности.

Замечание про параллелизм. На OCaml 4 истинной параллельности нет (`runtime-lock`), но конвейеризация через каналы прячет задержки; измеренное ускорение — $2.48\times$ на 4 воркерах. На OCaml 5 с `Domain` появляется реальный параллелизм, и тот же код пользуется им без изменений.

15. Что дальше

Вы прошли путь от потока-функции до `durable`-пайплайна с тревогами. Дальнейшие шаги:

- **Примеры.** Каталог `examples/` (`ex01-ex10`) идёт по возрастанию сложности; `ex07` — полноценный сервис мониторинга шахты с триангуляцией и retract-пайплайном газовых тревог.
- **Справочник API.** Интерфейсные файлы `.mli` (особенно `pipe.mli` и `trigger.mli`) написаны как самостоятельные руководства с разбором краевых случаев.
- **Глубже в семантику.** Статья `miniflink_article` разбирает устройство движка; `docs/orthogonal-persistence.md` — модель персистентности; `docs/atomic-update-event.md` — семантику `Update`.
- **Прикладной разбор.** Статья по `ex07` (`docs/ex07_article`) показывает, как из этих операторов собирается реальная система безопасности.

Главная мысль, которую стоит унести: пайплайн `miniFlink` читается как спецификация. Ключ объявлен один раз, время и поздние данные обработаны операторами, а решения о персистентности и параллелизме приняты снаружи и не засоряют логику. Это и есть та декларативность, ради которой всё затевалось.